

Figure 1: *Graph_3* generated by SageMath

adjacency matrix is symmetric along the diagonal. Line 6 checks if there is an edge between vertices i and j by examining the corresponding entry in the adjacency matrix. In case there is an edge, Line 7 adds that to the graph. Finally, Line 8 displays the graph that has been constructed.

Now, let us execute the code to view the output. As before, we use the online tool called CoCalc to do so. Log on to the CoCalc platform, type the code in the window, and save it as '*graph3.sagews*'. Then, press the Run button to execute the code. Figure 1 shows the graph called *Graph_3* generated by the above code on execution. It can be seen that *Graph_3* is isomorphic to *Graph_1* constructed in the previous article. Two graphs are called isomorphic if they are 'somewhat' equal. Please refer to the second article in this series for a more detailed explanation of isomorphic graphs.

Now, we construct a graph (which we will call *Graph_4*) using an incidence matrix with SageMath. The SageMath code, saved as '*graph4.sagews*', is provided below (line numbers have been included for clarity in the explanation).

```
1. from sage.graphs.graph import
   Graph
2. inc_matrix = [
       [1, 1, 1, 0, 0, 0],
       [1, 0, 0, 1, 0, 0],
       [0, 1, 0, 1, 1, 0],
       [0, 0, 1, 0, 0, 1],
       [0, 0, 0, 0, 1, 1]
   ]
3. G = Graph( )
```

```
4. num_vertices = len(inc_matrix)
5. num_edges = len(inc_matrix[0])
6. for i in range(num_vertices):
7.     G.add_vertex(i)
8.     for j in range(num_edges):
9.         edge=[ ]
10.        for i in range(num_vertices):
11.            if inc_matrix[i][j]
12.                == 1:
13.                    edge=edge+[i]
14.        G.add_
15.        edge(edge[0],edge[1])
16. G.show( )
```

Now, let us go through the code for better understanding. As before, Line 1 imports the Graph class from the *sage.graphs.graph* module. Line 2 stores the incidence matrix of the graph into the variable *inc_matrix*. Line 3 creates an instance of the Graph class, initialising an empty graph to the variable *G*. Line 4 calculates the number of vertices in the graph by finding the length of the incidence matrix, which is the number of rows. Line 5 calculates the number of edges in the graph by finding the length of the first row of the incidence matrix, which is the number of columns. The for loop in Line 6 iterates over the vertices of the graph. For each vertex index i , Line 7 adds a vertex to the graph. The for loop in Line 8 iterates over the edges of the graph. Line 9 initialises an empty list named *edge* to store the vertices incident to the current edge. The nested for loop in Line 10 iterates over the vertices of the graph again. The condition in Line 11 checks if vertex i is incident to edge j by examining the corresponding entry in the incidence matrix. If vertex i is incident to edge j , its index is added to the list named *edge* in Line 12. After identifying the two vertices incident to edge j , Line 13 adds an edge between them to the graph. Finally, Line 14 displays the constructed graph.

Now, let us execute the code to view the output. Use CoCalc to run the program named '*graph4.sagews*'. Figure 2 shows the graph called *Graph_4* generated by the above code on

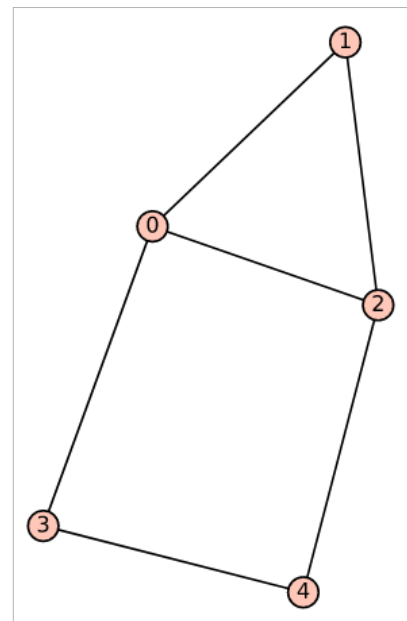


Figure 2: *Graph_4* generated by SageMath

execution. It can be seen that *Graph_4* is isomorphic to *Graph_2* constructed in the second article in the series.

Now that we are familiar with creating graphs using different methods that suit our needs, let us discuss some methods provided by SageMath to elicit more information from the graphs we have constructed. I will also introduce some graph theoretic terminology as and when required. In order to enhance clarity, I have occasionally opted for simplicity over strict mathematical rigour. As a result, some of the definitions provided may not adhere strictly to textbook standards. However, they should still offer you a good grasp of the concepts we are covering.

There are hundreds of methods offered by SageMath just for processing graphs alone, and we will be discussing a few so that you have an idea about how to use them. First, let us discuss the *all_paths()* method, which will list all the available paths between a pair of vertices. Formally, a *path* in a graph is a sequence of vertices where each vertex is connected to the next vertex by an edge. Add the following line of code, which will list all the paths from vertex